DAPLink 烧录器上手指南

——从装软件到打断点，一步一步把 STM32 调起来（小白手册）

本手册把原来零散的记录重新整理成一条完整流程：**先装好两个软件，再接好线，最后开两个终端敲命令。**每一条长命令都会拆开来讲清楚"每一段是什么意思"。照着做即可，遇到看不懂的名字可以随时往回翻。

本手册里出现的所有盘符、目录名、文件名（例如 E:\MyDAPLink_F103_精简版、out\MyDAPLink_BL.hex、interface\stlink.cfg）都是作者本机上的示例，你的电脑上很可能不一样。照着做时请务必：

- 把命令里的路径换成你自己文件的实际位置；
- 确认要烧录的 .hex 固件、要用的 *.cfg 配置文件确实存在于对应目录；
- 先在终端 cd 到正确目录再执行命令——路径对不上，会直接报「找不到文件」。

目录

1. 开始之前：先认识三个角色
2. 第一步：安装 OpenOCD
3. 第二步：安装 GDB（Arm GNU 工具链）
4. 第三步：连接硬件
5. 第四步：开两个终端，启动调试
6. 第五步：常用调试命令速查
7. 排错小贴士

1. 开始之前：先认识三个角色

这套流程里其实只有三样东西在配合工作：

- `OpenOCD`（Open On-Chip Debugger）：一个"翻译官"，负责把电脑上的调试命令翻译成芯片能听懂的 SWD 电信号。它需要一直开着，守在一个端口上等你来连。
- `GDB`：你在电脑上敲调试命令的地方。它把"下断点、单步、看变量"这些想法发给 OpenOCD。用在本项目里的是 ARM 版，叫 `arm-none-eabi-gdb`。
- `DAPLink 调试器`：一块小板子，一头插电脑 USB，一头用杜邦线连目标芯片。

电脑端是 Windows，所有命令都在 `PowerShell` 里执行（建议右键"以管理员身份运行"）。目标芯片是 STM32F103，上面已经烧好了程序（本手册示例文件是 `E:/LEDTEST.elf`）。

一句话流程：装软件 → 接线 → 开两个终端 → 敲命令。

2. 第一步：安装 OpenOCD

2.1 一键安装脚本

先打开 PowerShell，把下面整段复制进去回车执行（会自动下载安装并配好环境变量）：

```
# 1. 安装目录
$installDir = "C:\Tools\OpenOCD"
$tempDir = Join-Path $env:TEMP "OpenOCD-Install"
$zipPath = Join-Path $tempDir "openocd.zip"


# 2. 创建临时目录
New-Item -ItemType Directory -Force -Path $tempDir | Out-Null
New-Item -ItemType Directory -Force -Path $installDir | Out-Null


# 3. 通过 WinGet 安装 OpenOCD
winget install --id xpack-dev-tools.openocd-xpack --exact --accept-package-agreements --
accept-source-agreements


# 4. 查找安装后的 OpenOCD
$cmd = Get-Command openocd.exe -ErrorAction SilentlyContinue


if ($cmd) {
  $openocdDir = Split-Path $cmd.Source
} else {
  $candidates = @(
    "$env:LOCALAPPDATA\Programs\xPacks\@xpack-dev-tools\openocd",
    "$env:LOCALAPPDATA\xPacks\@xpack-dev-tools\openocd",
    "C:\Tools\OpenOCD"
  )


  $exe = Get-ChildItem $candidates -Filter openocd.exe -Recurse -ErrorAction
  SilentlyContinue |
    Select-Object -First 1


  if (-not $exe) {
    Write-Host "未自动找到 openocd.exe，请检查 WinGet 安装结果。" -ForegroundColor Yellow
    exit 1
  }


  $openocdDir = $exe.DirectoryName
}
```

```
# 5. 添加到当前用户 PATH（不需要管理员权限）
$userPath = [Environment]::GetEnvironmentVariable("Path", "User")
$paths = @($userPath -split ";" | Where-Object { $_ })

if ($openocdDir -notin $paths) {
  $newPath = (@($paths) + $openocdDir) -join ";"
  [Environment]::SetEnvironmentVariable("Path", $newPath, "User")
}

# 6. 当前 PowerShell 立即生效
if (($env:Path -split ";") -notcontains $openocdDir) {
  $env:Path += ";$openocdDir"
}

# 7. 验证
Write-Host "`nOpenOCD 安装目录：$openocdDir"
openocd --version
Write-Host "`nOpenOCD 路径："
where.exe openocd
```

2.2 安装成功的样子

看到类似的输出，就说明 OpenOCD 已经装好并进了 PATH：

3. 第二步：安装 GDB（Arm GNU 工具链）

`arm-none-eabi-gdb` 是 GDB 面向 ARM 芯片的版本，它随"Arm GNU 工具链"一起发布。同样打开 PowerShell，整段复制执行：

```
$ErrorActionPreference = "Stop"

# 1. 指定版本与安装目录
$version = "12.2.rel1"
$root = "C:\Tools\ArmGNU"
$zip = "$env:TEMP\arm-gnu-toolchain-$version.zip"
$url =
"https://armkeil.blob.core.windows.net/developer/Files/downloads/gnu/$version/binrel/arm-gnu-toolchain-$version-mingw-w64-i686-arm-none-eabi.zip"

# 2. 下载官方工具链 ZIP
Write-Host "正在下载 Arm GNU Toolchain $version ..."
Invoke-WebRequest -Uri $url -OutFile $zip
```

```
# 3. 解压到指定目录
New-Item -ItemType Directory -Force -Path $root | Out-Null
Expand-Archive -LiteralPath $zip -DestinationPath $root -Force

# 4. 自动查找 GDB 所在的 bin 目录
$gdb = Get-ChildItem -Path $root -Filter "arm-none-eabi-gdb.exe" -Recurse |
  Select-Object -First 1

if (-not $gdb) {
  throw "没有找到 arm-none-eabi-gdb.exe，请检查下载和解压结果。"
}

$bin = $gdb.DirectoryName
Write-Host "工具链目录：$bin"

# 5. 添加到当前用户 PATH
$userPath = [Environment]::GetEnvironmentVariable("Path", "User")
$paths = @($userPath -split ";" | Where-Object { $_ })

if ($bin -notin $paths) {
  $newPath = (@($paths) + $bin) -join ";"
  [Environment]::SetEnvironmentVariable("Path", $newPath, "User")
}

# 6. 当前窗口立即生效
if (($env:Path -split ";") -notcontains $bin) {
  $env:Path += ";$bin"
}

# 7. 验证安装
Write-Host "`nGDB 版本："
& $gdb.FullName --version

Write-Host "`nGDB 路径："
where.exe arm-none-eabi-gdb

Write-Host "`nGCC 版本："
arm-none-eabi-gcc --version
```

3.1 安装成功的样子

能看到 GDB 版本号（12.2.rel1）、GDB 路径和 GCC 版本，就说明工具链装好了：

4. 第三步：连接硬件

4.1 把 STM32 烧成 DAPLink 烧录器（用 ST-Link）

这块 DAPLink 烧录器不是买来的，是用一颗 STM32F103 自己"烧"出来的。固件已经编译好、放在本包的 out/ 文件夹里，你只要把它写进芯片就行。这一步做完，才有第 5 章里那个能用的烧录器。

动手前先备齐：

- 一块 STM32F103——必须 **128KB flash**（也就是 CB / 128K 版本）；64KB 的 C8 装不下完整固件。
- 一个 ST-Link 下载器，用 SWD 接在芯片的 PA13 / PA14 上。
- 本包里的两个固件：`MyDAPLink_BL.hex`（引导固件 BL）和 `MyDAPLink_IF_MSC.hex`（接口固件 IF）——两个都要烧，缺一不可。

第 1 步：先确认芯片够不够大

```
openocd -f interface/stlink.cfg -f target/stm32f1x.cfg -c "init; flash probe 0; exit"
```

看到 `flash size = 128 KiB`，就说明这颗芯片够用；如果是 64 KiB，就得换一块 CB 的板子。

第 2 步：烧录两个固件

```
cd "E:\MyDAPLink_F103_精简版"
openocd -f interface/stlink.cfg -f target/stm32f1x.cfg -c "adapter speed 4000" -c "init; reset halt; flash erase_sector 0 0 last; program out/MyDAPLink_BL.hex verify; program out/MyDAPLink_IF_MSC.hex verify reset exit"
```

注意：这里用的是 `interface/stlink.cfg`（拿 ST-Link 去烧）。本包 README 里那条命令写的是 `interface/cmsis-dap.cfg`，那是拿"一块已经做好的 DAPLink"去烧另一块用的，别搞混。

顺带说一句：这版固件特意保留了芯片自身的 SWD（PA13 / PA14），所以烧完之后 ST-Link 照样能连、可以反复重烧，不会变砖。

烧录成功的标志：两段固件都出现 `Programming Finished` 和 `Verified OK`，最后自动复位：

把 DAPLink 调试器用 USB 线插到电脑，再用杜邦线把它的调试脚连到目标板：

4.2 引脚接线

按下表连接调试器与目标板 STM32F103：

DAPLink（本固件）	目标板 STM32F103	说明
PB6（SWCLK）	PA14（SWCLK）	SWD 时钟
PB7（SWDIO）	PA13（SWDIO）	单脚双向 SWDIO
PB8（nRESET）	NRST	可选
GND	GND	必须共地

提示：目标板需单独供电 3.3V；GND 必须共地；nRESET 不接也能调试。

为什么调试器的调试脚是 PB6 / PB7 / PB8，而不是常见的 PA13 / PA14？因为这块调试器的固件把 SWD 输出改到了 PB6/PB7/PB8，把 PA13/PA14 让给了芯片自己；而目标板 STM32F103 的 SWD 固定就在 PA14/PA13。所以上表是「按功能对接」(左边的 SWCLK 接右边的 SWCLK)，两边脚号不同属于正常现象。

4.3 装驱动：让电脑认出 DAPLink（Zadig，只需做一次）

DAPLink 插上电脑后，Windows 会先用系统自带的 HID 驱动去认它。这块板子的固件是 **CMSIS-DAP v2（bulk 传输)**：HID 驱动能认到设备，但后面跑 OpenOCD 调试命令会失败。所以**第一次使用必须先换一次驱动**（只需做一次，之后插拔都有效)。

1. 只把 DAPLink 用 USB 线插到电脑上，先不接目标板。
2. 打开 Zadig（https://zadig.akeo.ie/)，勾上菜单里的 `Options → List All Devices`。
3. 在下拉列表里选到 **DAPLink CMSIS-DAP**，核对它右侧显示的 VID/PID 是 `0D28` / `0204`。
4. 把右侧要替换的驱动选成 **libusbK**，点 **Replace Driver**，等进度条走完。
5. 完成后设备会断开重连一次，属正常现象。可在 PowerShell 里执行 `Get-PnpDevice -PresentOnly | Where-Object { $_.InstanceId -like '*VID_0D28*' }` 确认它还在。

重要：务必核对 VID/PID = `0D28` / `0204`。 这两个值是固件里写死的（ARM mbed 的 DAPLink CMSIS-DAP 身份)，选错了会把键盘、鼠标等其它 USB 设备的驱动换掉。

漏做这一步的典型症状：下一步运行 OpenOCD 时提示 `unable to find a matching CMSIS-DAP device`。

5. 第四步：开两个终端，启动调试

为什么要开**两个**窗口？因为 OpenOCD 要一直开着当"后台服务"守在端口上，再由 GDB 连到它上面去。所以：窗口一跑 OpenOCD，窗口二跑 GDB。

5.1 窗口一：启动 OpenOCD

```
openocd -f interface/cmsis-dap.cfg -f target/stm32f1x.cfg -c "transport select swd" -c "adapter speed 1000" -c "cmsis-dap vid_pid 0x0D28 0x0204" -c "cmsis-dap backend usb_bulk" -c "cmsis-dap usb interface 4"
```

逐段拆解：

- `openocd` —— 要运行的程序名。
- `-f interface/cmsis-dap.cfg` —— `-f` 表示"加载一个配置文件"；这一份描述**调试器接口**（`cmsis-dap` 就是 DAPLink 使用的协议)。
- `-f target/stm32f1x.cfg` —— 再加载一份配置，描述**目标芯片**（STM32F1 系列)。
- `-c "transport select swd"` —— `-c` 表示"执行一条命令"；这条选择 `swd` 传输方式（比 JTAG 占用的

引脚更少)。

- `-c "adapter speed 1000"` —— 设置调试通信速率为 1000 kHz。如果连接不稳定，可以把数字调小，比如 500。
- `-c "cmsis-dap vid_pid 0x0D28 0x0204"` —— 指定只认这个 VID/PID 的 CMSIS-DAP 设备。电脑上若同时有多个调试器（或别家厂商的 CMSIS-DAP 板），不加这个会选错设备。
- `-c "cmsis-dap backend usb_bulk"` —— 改用 bulk 传输后端，对应 4.3 装好的 libusbK 驱动。漏掉它会退回默认的 HID/WinUSB 后端，典型症状是能连上、但命令超时（CMD_INFO 5s timeout）。
- `-c "cmsis-dap usb interface 4"` —— 指定 bulk 数据所在的 USB 接口号，这块固件是 **4**。写错或漏写都会报找不到设备。

运行后，窗口里出现 `Listening on port 3333 for gdb connections`，就说明服务起来了，别关这个窗口：

5.2 窗口二：启动 GDB 并连接

另开一个 PowerShell 窗口，执行：

```
arm-none-eabi-gdb E:/LEDTEST.elf -ex "target extended-remote localhost:3333" -ex "monitor reset halt" -ex "break main" -ex "continue"
```

逐段拆解：

- `arm-none-eabi-gdb` —— 要运行的程序（ARM 版 GDB）。
- `E:/LEDTEST.elf` —— 要调试的固件文件。`.elf` 里带着调试信息（变量名、行号等），所以调试要用它而不是 `.hex`。
- `-ex "命令"` —— 启动后**自动执行一条命令**，可以写多次，按顺序依次执行。下面四条就是这么来的。
- `"target extended-remote localhost:3333"` —— 连接到本机 3333 端口上的 OpenOCD，也就是窗口一开的那个服务。
- `"monitor reset halt"` —— 让芯片复位并停下来。`monitor` 开头的命令是"交给 OpenOCD 去硬件上执行"的。
- `"break main"` —— 在 `main` 函数处下一个断点。
- `"continue"` —— 让程序跑起来，到断点会停下。

连接成功后，会看到 `Breakpoint 1, main () at ...`，光标停在 `(gdb)` 提示符上等待输入：

6. 第五步：常用调试命令速查

下面的命令都在 **GDB 窗口**里输入，也就是在 `(gdb)` 提示符后面敲。

6.1 运行控制

- `continue`（可简写 `c`）：继续运行程序，遇到断点才停下。
- `monitor reset halt`：复位并暂停芯片，相当于"重新来过并停在起点"。
- `next`（可简写 `n`）：单步执行一行；遇到函数调用**不进去**，直接把它当一步走完。
- `step`（可简写 `s`）：单步执行一行；遇到函数调用**会进去**，进到函数内部。

6.2 断点

- `break main`：在 main 函数下断点。也可以写成 `break 文件名:行号`，例如 `break main.c:24`。
- `info breakpoints`：列出当前所有断点（编号、地址、位置等）。
- `delete 1`：删除编号为 1 的断点；如果只写 `delete`（不带编号）则删除全部，会再让你确认一次。

6.3 查看信息

- `print 变量名`（可简写 `p`）：打印某个变量的值，例如 `print uwTick`。
- `bt`（backtrace）：查看函数调用栈，即"现在是被谁一路调用到这里的"。
- `info registers`：查看 CPU 各个寄存器当前的值。

7. 排错小贴士

- **提示"不是内部或外部命令"**：多半是软件没进 PATH。先关掉终端重开一个新窗口再试；还不行就检查安装脚本的最后一步是否成功。
- **OpenOCD 连不上芯片**：依次检查 ① 杜邦线是否接对（SWCLK / SWDIO / GND）、② 目标板是否单独供电、③ 把速率调小试试，例如 `adapter speed 500`。
- 提示 `unable to find a matching CMSIS-DAP device`：先回到 **4.3** 确认 Zadig 驱动已经换过；再检查 **5.1** 的命令里 `cmsis-dap vid_pid 0x0D28 0x0204`、`cmsis-dap backend usb_bulk`、`cmsis-dap usb interface 4` 这三个参数有没有漏写 —— 手敲命令时最容易丢掉的就是这三个。
- **命令莫名失败**：检查是不是混进了中文全角符号——`–`（破折号）、`"`（中文引号）、`：`（中文冒号）都会让命令失效，务必换成英文半角。
- **GDB 报 `No such file or directory`**：它只是找不到 `.c` 源代码文件，不影响单步和断点，可以忽略。
- **断点加不上 / 数量超限**：STM32F103 一般只有 6 个硬件断点，用 `delete` 删掉不用的再试。

OpenOCD 报 cannot read IDR：多半是目标板没供电或没共地。先确认目标板单独接了 3.3V、且和调试器 GND 连在一起，再检查 SWD 两根线有没有接反。

- **想重来一遍**：先关掉两个终端窗口，重新按第 5 步开两个窗口即可。

附录 A：这块 DAPLink 烧录器是怎么来的

本手册用的烧录器不是买来的成品，而是用 ARM 官方的 DAPLink 固件改出来的：源码取自 GitHub 的 ARMmbed/DAPLink（main 分支），原本面向 STM32F103RB（NUCLEO 板），被移植到了常见的 Blue Pill（STM32F103）上。相对原版做了三处关键改动：

① 保留芯片自身的 SWD 调试口：把 SWJ_DISABLE 改成 SWJ_NOJTAG，烧完固件后芯片仍能用 SWD 重新烧写，不会「变砖」。

② 把调试输出脚从 PA13/PA14 挪到 PB6/PB7/PB8：把 PA13/PA14 让给芯片自己，这也正是接线表里调试器一侧脚号、与目标板一侧不同的原因。

③ 把 5 个指示灯统一到板载 LED（PC13）：接口工作时亮，拖拽烧录时快速闪烁，方便用肉眼判断状态。

还有一个和选型有关的点：官方 DAPLink 固件需要 128KB flash，所以要选 STM32F103CB（128KB）；64KB 的 C8 装不下完整功能。

附录 B：重刷固件 / 变砖救回

B.1 正常升级（拖拽烧录）

烧录器插上电脑后会多出一个虚拟 U 盘（例如 D:）。升级固件不用任何工具，直接往这个盘里拖文件即可：

```
Copy-Item out\magic\START_BL.ACT D:\
Copy-Item out\MyDAPLink_IF_MSC.hex D:\
Get-Content D:\DETAILS.TXT
```

两个必须注意的坑：① 文件必须叫 START_BL.ACT（带点号）——写成 START_BLACT 会被 Windows 改成短名，命令永远不会生效；② 拖完不要拔 USB——固件靠软复位切换模式，一拔就断电，切换标志会丢失。

B.2 变砖救回（串口 ISP）

如果一块板子被烧进了关闭 SWD 的固件、用 SWD 怎么都连不上，可以用 BOOT0 + 串口 ISP 把它擦掉救回。

接线：USB-TTL（CH340）的 TXD→PA10、RXD→PA9、GND→GND；目标板把 BOOT0 接 3.3V（BOOT1=0），并且先设好 BOOT0 再上电（复位瞬间才会采样）。

```
python rescue\isp_rescue.py <固件.bin> COM16
```

关键坑：串口必须用偶校验（8E1）；用默认的 8N1 会握手通过、但之后所有命令全部失败。